Electrical and Computer Engineering Department

Artificial Intelligence- ENCS3340

First Semester, 2022-2023

Project 1

Department Courses Distribution

Prepared By:

Ayah Saad 1191334

Hala Ziq 1191637

Farha Saad 1191331

Instructor:

Aziz Qaroush

Abstract

The Department Courses Distribution project discusses an interesting and important topic for all Birzeit students, the aim of this project is to use Genetic algorithm to optimize the distribution of the courses to achieve maximum diversity, and this problem has many constraints. A convenient way to do this model is to use a genetics algorithm

A genetic algorithm is a search heuristic that is inspired by Charles Darwin's theory of natural evolution. This algorithm reflects the process of natural selection where the fittest individuals are selected for reproduction in order to produce offspring of the next generation. [1]

Table of Contents

Abstract 1

Introduction 3

Problem Formulation 4

Design 6

Conclusion 8

References 9

Introduction

Developing a department courses distribution is not an easy task. This problem has become one of the most important and hard problems to solve in university. The task has many variables: time slots, courses, sections, student priorities, teacher priorities etc. The aim of this project is to distribute the department courses with max diversity. We tried to optimize this problem by Genetic algorithm which based mainly on the fitness function that can decrease the population, just take good chromosomes. With Genetic algorithm we will use Generate population of solutions, crossover and mutation.

Problem Formulation

Generally, the Genetics Algorithm has

1. Genes
2. Chromosomes
3. Populations.

   Here the chromosomes have fitness functions, by crossing over and mutating the chromosomes we'll get new chromosomes with better fitness, and this is how we got a population. The goal from the crossover and mutation is to make Solution optimization or to get zero conflicts.

✓ Solution:

It formalized as Schedule, size of it as number of courses, every element in array must assign with course id, course name, instructor name and meeting time

✓ Generate population

We generated number of solutions, this number is initializing in the class.

✓ Fitness function:

It depends on a lot of factors mention below:

- First, for the same Department and the same Meeting Time, the number of conflicts increase in 3 cases:
  1. some course taught at the same time
  2. one doctor who gives different subject at the same time
  3. Different subjects from the same year of studying

- Second, for the two different Departments and the same Meeting Time, the number of conflicts increase in one case:

1. the common courses

- Third, for the same Department and the same Meeting Lab Time, the number of conflicts increase in 3 cases:
    1. for some course taught at the same time
    2. for one doctor who gives more than one subject
    3. for Different subjects but from the same year of studying

- Fourth, for the same Departments and the same time of Lecture and lab, the number of conflicts increase in 3 cases:
    1. Lab and lecture time on the same day
    2. The time of the Lab and lecture is for the same doctor
    3. The time of the Lab and lecture is for the same year of studying

- Fifth, for the Different Departments and the same time of Lecture and lab, the number of conflicts increase in 3 cases:
    1. Lab and lecture time on the same day
    2. Common course from the same year of studying
    3. Common course for the same doctor

✓ Crossover

In crossover, after we select good solution from population depend on fitness, we crossover two parents by take random part from parent1 and put it on child, and complete child from parent2, taking into account that no repetition in elements in child chromosome. We produce new generations with assumption that good parents produce better child than him.

Programing:

We've initialized a crossover-rate = 0.9, this crossover arising when crossover-rate is larger than the random value number.

- ✓ Mutation

  We use mutation when the fitness of solutions does not increase; so we've initialized a mutation-rate which are very low, around **0.1**and this mutation only occurs when it is greater than the random value.

Design

In our design the fitness function is calculated by division (1/summation of conflicts). Based on the fitness function of each chromosome the population is sorted.

We built the needed classes all, which are:

Instructor [Name],

Course [Name, id, years, Instructor, Number of hours, Number of Sections],

Schedule [Classes, FitnessChange, Fitness, ClassNum, NumOfConflict, Data],

Class [id, Department, course, Instructor, MeetingTime],

MeetingTime [id, Time],

Department [name, courses],

Data [Instructor, Course, Department, MeetingTime, Number of Classes],

GeneticAlgorithm,

Population [Schedules],

Driver

Output:

```
-->   Generation # 0
_____________________________________________________________________________________________________
_____________
_____________________________________________________________________________________________________
_____________


 We Make A Population with 20 Schedule (Chromosome)
 For Each Schedule (Chromosome) We Calculate The Fitness and # of Conflict
 #Schedule|Fitness|#Conflicts
    0 | 0.00308 | 324
    1 | 0.00304 | 328
    2 | 0.00302 | 330
    3 | 0.00299 | 334
    4 | 0.00296 | 337
    5 | 0.00295 | 338
    6 | 0.00292 | 342
    7 | 0.00287 | 347
    8 | 0.00287 | 348
    9 | 0.00284 | 351
    10 | 0.00284 | 351
    11 | 0.00282 | 353
    12 | 0.00279 | 358
    13 | 0.00278 | 359
    14 | 0.00277 | 360
    15 | 0.00276 | 361
    16 | 0.00275 | 362
    17 | 0.00273 | 365
    18 | 0.00272 | 367
    19 | 0.00269 | 371
```

```
                    #Schedule: 0 Class # |          Dept     |     Course (id)     |       Instructor (n
ame)  |          Meeting-Time (time)

_____________________________________________________________________________________________________
_____________
                01          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Jamal Seyam] | M 11:25-14:05
                02          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Ibrahim Nemer] | T 11:25-14:05
                03          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Ismail khater] | T 2:15-16:55
                04          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Adnan Yahya] | W 11:25-14:05
                05          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Ismail khater] | T 8:00-10:40
                06          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Adnan Yahya] | R 8:00-10:40
                07          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Khader Mohammad] | S 2:15-16:55
                08          | Computer Engineering        | DIGITAL ELECTRONICS AND COMPUTER ORGANIZAT
ION LABORATORY (ENCS2110          | [Bilal Karaki] | T 11:25-14:05
                09          | Computer Engineering        | DIGITAL SYSTEMS (ENCS2340     | [Ismail kh
ater] | TR 10:00-11:15
                10          | Computer Engineering        | DIGITAL SYSTEMS (ENCS2340     | [Ismail kh
ater] | SM 12:50-2:05
                11          | Computer Engineering        | DIGITAL SYSTEMS (ENCS2340     | [Mohammed
Hussein] | TR 12:50-2:05
                12          | Computer Engineering        | DIGITAL SYSTEMS (ENCS2340     | [Bilal Kar
aki] | SW 11:25-12:40
                13          | Computer Engineering        | DIGITAL SYSTEMS (ENCS2340     | [Anjad Bad
```

Conclusion

We noticed after the experience of genetic algorithm to solve the problem, that we reach a good solution, but it depends on a lot of factors, specifically on fitness function, this is more accurate if we takes into account all the elements that affect the problem, and reach better solution with shorter time, as well as the solution depends on the sample size, if we take larger population it took more time to finish but it starts from good solutions because of its diversity due to its larger. In general genetic algorithm is slow to find a solution, and we can apply other way to get the solution faster.

References

1. https://towardsdatascience.com/introduction-to-genetic-algorithms-including-example-code-e396e98d8bf3